

Tutorial Interactivo: Nuevas Funcionalidades de FEISS con APIs Integradas y Web Categorizada

¡Bienvenido al tutorial interactivo de las nuevas y potentes funcionalidades de FEISS! Hemos integrado una serie de APIs adicionales y hemos reorganizado la estructura de la web para ofrecerte una experiencia más intuitiva y completa.

Este tutorial te guiará a través de las principales interacciones y cómo puedes aprovechar al máximo estas integraciones y la nueva organización de la web.

Nueva Estructura de la Web de FEISS

La web de FEISS ahora está organizada en las siguientes categorías principales para una navegación más clara:

- **Inicio:** La página principal con una visión general de FEISS.
- **Servicios:** Detalles de las soluciones que ofrece FEISS.
- **Integraciones API:** Información sobre todas las APIs conectadas.
- **Documentación:** Acceso a guías, tutoriales y la base de conocimiento.
- **Contacto:** Formulario para consultas y soporte.

1. Proceso de Compra Mejorado y Automatización Post-Venta

El proceso de compra ahora es más robusto y está conectado con sistemas de automatización.

Escenario: Un cliente compra un plan en FEISS.

Interacción:

1. El cliente completa el formulario de pago en la web.
2. El servidor procesa el pago a través de **Stripe**.
3. **Gmail API** envía automáticamente un correo de confirmación de compra personalizado al cliente.
4. **Zapier API** activa un flujo de trabajo post-venta que puede incluir:
 - Añadir al cliente a tu CRM (ej. HubSpot, Salesforce).
 - Suscribir al cliente a una lista de correo (ej. Mailchimp).
 - Enviar una notificación al equipo de ventas en **Slack**.

Cómo probarlo (simulado): Para simular este flujo, puedes hacer una llamada `POST` a la ruta `/api/create-payment-intent` en tu servidor. Asegúrate de que tu archivo `.env` tenga configuradas las claves de Stripe, Gmail y Zapier.

```
curl -X POST http://localhost:3001/api/create-payment-intent \
  -H "Content-Type: application/json" \
  -d
'{
  "paymentMethodId": "pm_card_visa",
  "amount": 5000,
  "email": "nuevo.cliente@example.com",
  "fullName": "Carlos García",
  "company": "Tech Solutions",
  "planId": "professional",
  "planName": "Professional"
}'
```

Verificación:

- Revisa la bandeja de entrada de `nuevo.cliente@example.com` para el correo de confirmación.
- (Si configurado en Zapier) Verifica tu CRM, lista de correo o canal de Slack para las notificaciones.

2. Reserva de Demostraciones con Google Calendar y Cal.com

Ahora los usuarios pueden reservar demostraciones directamente, y estas se gestionan automáticamente en tu calendario.

Escenario: Un usuario quiere agendar una demo de FEISS.

Interacción:

1. El usuario selecciona una fecha y hora disponible en la web.
2. El servidor utiliza la **Google Calendar API** o **Cal.com API** para crear un evento en tu calendario.
3. Se envía una invitación al correo del usuario con los detalles de la demo.

Cómo probarlo (simulado): Haz una llamada `POST` a la ruta `/api/book-demo` (para Google Calendar) o `/api/cal-com/create-booking` (para Cal.com).

```
# Para Google Calendar
curl -X POST http://localhost:3001/api/book-demo \
  -H "Content-Type: application/json" \
  -d
'{
  "email": "interesado@example.com",
  "name": "Ana López",
  "date": "2026-06-30",
  "time": "10:00"
}'

# Para Cal.com
curl -X POST http://localhost:3001/api/cal-com/create-booking \
  -H "Content-Type: application/json" \
  -d
'{
  "eventType": "demo_feiss",
  "email": "interesado@example.com",
  "name": "Ana López",
  "date": "2026-07-01",
  "time": "11:00"
}'
```

Verificación:

- Revisa tu Google Calendar o Cal.com para el nuevo evento.
- Revisa la bandeja de entrada de `interesado@example.com` para la invitación.

3. Documentación Dinámica y Base de Conocimiento con Notion

Tu documentación y base de conocimiento ahora se gestionan de forma centralizada en Notion y se muestran dinámicamente en la web.

Escenario: Un usuario busca información sobre cómo usar FEISS.

Interacción:

1. El usuario introduce una consulta en la sección de documentación de la web.
2. El servidor utiliza la **Notion API** para buscar contenido relevante en tus bases de datos de Notion.
3. El contenido se renderiza directamente en la web, asegurando que siempre esté actualizado.

Cómo probarlo (simulado): Haz una llamada `GET` a la ruta `/api/documentation` con un parámetro `query`.

```
curl "http://localhost:3001/api/documentation?query=instalacion"
```

Verificación:

- La respuesta JSON contendrá los resultados de la búsqueda en Notion.

4. Gestión de Tickets de Soporte con Notion y Gmail

El proceso de soporte ahora es más eficiente, registrando las consultas y confirmando su recepción.

Escenario: Un usuario envía una consulta de soporte.

Interacción:

1. El usuario rellena el formulario de soporte en la web.
2. El servidor utiliza la **Notion API** para crear una nueva página (ticket) en tu base de datos de soporte de Notion.
3. La **Gmail API** envía un correo de confirmación al usuario, indicando que su ticket ha sido recibido.

Cómo probarlo (simulado): Haz una llamada `POST` a la ruta `/api/support-ticket`.

```
curl -X POST http://localhost:3001/api/support-ticket \
  -H "Content-Type: application/json" \
  -d
'{
  "email": "soporte@example.com",
  "name": "Pedro Ruíz",
  "subject": "Problema con la configuración",
  "message": "No consigo configurar el módulo X. ¿Podrían ayudarme?"
}'
```

Verificación:

- Revisa tu base de datos de soporte en Notion para el nuevo ticket.
- Revisa la bandeja de entrada de `soporte@example.com` para el correo de confirmación.

5. Integraciones Adicionales (Alpha Vantage, Amplitude, Apify, Consensus, GitHub, Hume, Instagram, Lumin PDF, Meta Ads, MotherDuck, Neimo, Outlook Calendar, Outlook Mail, Scite, Vercel)

Hemos preparado el terreno para estas APIs, creando los servicios y las rutas básicas en `server_enhanced.js`.

Interacción General: Cada una de estas APIs tiene un servicio (`.js` en la carpeta `services/`) y una ruta de ejemplo en `server_enhanced.js`.

- **Alpha Vantage:** `/api/alpha-vantage/stock-data` (GET) - Datos financieros.
- **Amplitude:** `/api/amplitude/track-event` (POST) - Análisis de producto y marketing.
- **Apify:** `/api/apify/run-actor` (POST) - Extracción de datos web y automatización.
- **Consensus:** `/api/consensus/search-papers` (GET) - Búsqueda de artículos científicos.
- **GitHub:** `/api/github/repo-info` (GET) - Control de versiones y gestión de repositorios.
- **Hume AI:** `/api/hume/analyze-emotion` (POST) - Análisis de emociones en texto.
- **Instagram:** `/api/instagram/user-posts` (GET) - Obtener publicaciones de un usuario.
- **Lumin PDF:** `/api/lumin-pdf/process-document` (POST) - Procesar documentos PDF.
- **Meta Ads Manager:** `/api/meta-ads/campaign-insights` (GET) - Datos de campañas publicitarias.
- **MotherDuck:** `/api/motherduck/query-data` (POST) - Ejecutar consultas SQL en bases de datos analíticas.
- **Neimo:** `/api/neimo/regulatory-info` (GET) - Información regulatoria.
- **Outlook Calendar:** `/api/outlook-calendar/create-event` (POST) - Gestión de calendario.
- **Outlook Mail:** `/api/outlook-mail/send-email` (POST) - Gestión de correo electrónico.
- **Scite:** `/api/scite/citations` (GET) - Búsqueda de citas científicas.
- **Vercel:** `/api/vercel/deployments` (GET) - Información sobre tus despliegues.

Cómo usar: Deberás implementar la lógica específica dentro de cada función de servicio (`sendData`, `fetchData`) para interactuar con la API correspondiente según su documentación. Las rutas en `server_enhanced.js` ya están configuradas para llamar a estos servicios.

Ejemplo (Alpha Vantage):

```
curl "http://localhost:3001/api/alpha-vantage/stock-data?symbol=IBM"
```

Verificación:

- La respuesta JSON mostrará los datos de la acción (una vez implementada la lógica interna del servicio de Alpha Vantage).

Próximos Pasos

1. **Configura tus variables de entorno:** Asegúrate de que todas las `API_KEY` y `BASE_URL` estén correctamente definidas en tu archivo `.env` para cada servicio que desees utilizar.
2. **Implementa la lógica específica:** Para las APIs adicionales, personaliza las funciones `sendData` y `fetchData` en sus respectivos archivos de servicio (`services/*.js`) para que realicen las operaciones deseadas con la API externa.
3. **Desarrolla el Frontend:** Crea la interfaz de usuario en tu `index_categorized.html` o en tu framework de frontend (React, Vue, etc.) para interactuar con estas nuevas rutas API.

¡Esperamos que disfrutes de estas nuevas capacidades y de la web organizada! Si tienes alguna pregunta, no dudes en consultar la `MCP_INTEGRATION_GUIDE.md` o contactar al equipo de soporte.